

Replication Plan

Deep Learning of Atomically Resolved Scanning Transmission Electron Microscopy
Images: Chemical Identification and Tracking Local Transformations

OSTI ID: 1427646

Auto-generated Replication Blueprint

April 8, 2026

Contents

1	Paper Overview	2
2	Detailed Workflow	2
2.1	Phase 1: Simulated Training Data Generation	2
2.2	Phase 2: FCN Architecture and Training	2
2.3	Phase 3: Application to Experimental Data	3
2.4	Phase 4: Analysis and Validation	3
3	Datasets	3
4	Models, Tools, and Software	3
4.1	Programming Languages	3
4.2	Libraries and Frameworks	4
4.3	Hardware Requirements	4

Paper Overview

This paper applies deep learning—specifically fully convolutional neural networks (FCNs)—to atomically resolved scanning transmission electron microscopy (STEM) images. The method performs pixel-level chemical identification of atomic columns in complex oxide heterostructures (e.g., $\text{SrTiO}_3/\text{La}_{0.7}\text{Sr}_{0.3}\text{MnO}_3$ interfaces), identifying atom types (Sr, Ti, La/Sr, Mn, O, vacuum) and tracking local structural/chemical transformations. Training uses simulated STEM images generated via multislice calculations, and the trained model is then applied to experimental STEM data.

Detailed Workflow

Phase 1: Simulated Training Data Generation

1.1 Build atomic structure models.

- Construct supercells of SrTiO_3 and $\text{La}_{0.7}\text{Sr}_{0.3}\text{MnO}_3$ perovskites in the $[001]$ zone axis.
- Introduce controlled defects: vacancies, anti-site substitutions, column intermixing.
- Vary supercell thickness (number of unit cells along beam direction).

1.2 Run multislice STEM image simulations.

- Use the multislice algorithm (e.g., via PRISM or Dr. Probe) to simulate HAADF-STEM and ABF-STEM images.
- Parameters: beam energy (typically 200 keV), convergence semi-angle, detector angles, probe step size, specimen thickness.
- Add Poisson noise and scan distortions to simulate realistic experimental conditions.

1.3 Generate ground-truth label maps.

- For each simulated image, create a pixel-wise label map assigning each pixel to an atom type or background class.
- Use Gaussian-weighted labels centered on atomic column positions.

Phase 2: FCN Architecture and Training

2.1 Implement the fully convolutional network.

- Encoder path based on VGG-like blocks (convolution $\rightarrow$ batch norm $\rightarrow$ ReLU $\rightarrow$ pooling).
- Decoder path with transposed convolutions / bilinear upsampling and skip connections.
- Final softmax layer producing per-pixel class probabilities.

2.2 Data augmentation.

- Random rotations, flips, cropping, brightness/contrast perturbations.
- Elastic deformations to simulate scan distortions.

2.3 Train the FCN.

- Loss function: cross-entropy loss.
- Optimizer: Adam with learning rate scheduling.
- Train/validation split from simulated images; monitor validation loss.

Phase 3: Application to Experimental Data

3.1 Preprocess experimental STEM images.

- Normalize intensity, correct scan drift artifacts.
- Crop to regions of interest.

3.2 Apply the trained FCN to produce per-pixel classification maps.

3.3 Post-processing.

- Extract atomic column positions from probability maps (peak finding).
- Assign chemical identity to each column based on maximum class probability.

Phase 4: Analysis and Validation

4.1 Quantitative evaluation on simulated test data.

- Pixel-level accuracy, per-class precision/recall, confusion matrix.
- Positional accuracy of detected atomic columns (RMSE).

4.2 Reproduce key figures.

- Overlay classification maps on STEM images.
- Generate line profiles of composition across interfaces.
- Track compositional changes in time-series STEM data.

Datasets

Dataset / Source			Type	Location / Notes
Simulated STEM images			Synthetic (multi-slice)	Generated using PRISM or Dr. Probe from crystal structure files
Crystal structure files (CIF)	structure files		Reference structures	https://materialsproject.org/ for STO, LSMO structures; also https://www.crystallography.net/cod/
Experimental STEM images	STEM images		Experimental	Available from the authors or Oak Ridge National Laboratory data archives; some may be in supplementary information
Supplementary information	infor- mation		Tables, images	Accompanies the paper; contains additional classification maps and parameters

Models, Tools, and Software

Programming Languages

- **Python 3.8+** — primary language for deep learning and image processing.

Libraries and Frameworks

Tool / Library	Version	Purpose / URL
PyTorch	≥ 1.10	Deep learning framework for FCN https://pytorch.org/
torchvision	≥ 0.11	Pre-trained encoder backbones, data augmentation
NumPy / SciPy	≥ 1.21 / ≥ 1.7	Numerical operations, image processing
scikit-image	≥ 0.18	Image preprocessing, peak finding, filtering
Matplotlib	≥ 3.4	Visualization of classification maps and overlays
PRISM (Prismatic)	latest	GPU-accelerated multislice STEM simulation https://prism-em.com/
abTEM	latest	Alternative multislice simulator in Python https://abtem.readthedocs.io/
Dr. Probe	latest	STEM image simulation https://er-c.org/barthel/drprobe/
ASE (Atomic Simulation Environment)	≥ 3.22	Crystal structure building https://wiki.fysik.dtu.dk/ase/
Atomai	latest	Deep learning for microscopy (from ORNL) https://github.com/pycroscopy/atomai

Hardware Requirements

- **GPU:** NVIDIA GPU with ≥ 8 GB VRAM for FCN training (e.g., RTX 3080, V100).
- **RAM:** ≥ 16 GB for multislice simulations and large image datasets.
- **Storage:** ~ 50 – 100 GB for simulated training image library.